

FinSight

Regulatory-Grade Financial Intelligence System

Technical System Documentation

Generated Technical Reference

July 16, 2026

Abstract

FinSight is a multi-agent Retrieval-Augmented Generation (RAG) system for analyzing corporate financial filings (annual reports) with an explicit design mandate for *compliance-grade trustworthiness* rather than conversational convenience. Every claim surfaced to the user is traceable to a verbatim source citation (document, page, snippet); claims that cannot be verified against retrieved evidence are hard-blocked before reaching the output, not merely flagged. The system decomposes a user query into subtasks via a Planner agent, retrieves evidence through a hybrid dense/sparse retrieval pipeline, extracts candidate claims via an Analyst agent, verifies each claim against its source through an entailment-checking Auditor agent, performs deterministic cross-document comparison through a Comparator agent, and finally synthesizes a structured Markdown report through a Synthesizer agent. Every run produces a machine-readable audit log capturing the full decision trail. This document provides an exhaustive technical reference to the codebase: architecture, module-by-module responsibilities, data models, API surface, configuration, testing and evaluation methodology, and deployment topology.

Contents

1	Introduction and Project Purpose	3
1.1	Overview	3
1.2	Design Mandate	3
1.3	Confidence Model	3
2	System Architecture	5
2.1	Orchestration Framework	5
2.2	The Six-Stage Pipeline	5
2.3	Context Propagation	6
2.4	Concurrency and Latency Management	7
3	Repository Layout	8
4	Technology Stack	10
4.1	Core Dependencies	10
4.2	External Services	10
4.3	Frontend	11
5	Module Reference	12
5.1	Agents (agents/)	12
5.1.1	router.py — PlannerAgent	12
5.1.2	retriever.py — RetrieverPlugin	12
5.1.3	analyst.py — AnalystPlugin	12
5.1.4	auditor.py — AuditorPlugin	13
5.1.5	comparator.py — ComparatorPlugin	13
5.1.6	synthesizer.py — SynthesizerAgent	13
5.2	Core Infrastructure (core/)	14
5.2.1	config.py	14
5.2.2	models.py	14
5.2.3	sk_kernel.py	14
5.2.4	groq_client.py	14
5.2.5	unit_normalizer.py	14
5.3	Retrieval Pipeline (retrieval/)	15
5.3.1	Confidence Scoring	15
5.4	Ingestion Pipeline (ingestion/)	15
5.5	Observability (observability/tracer.py)	16
6	Data Models	17
6.1	Enumerations	17
6.2	Core Dataclasses	17
6.3	Qdrant Collection Schema	18
6.4	Audit Log Example	19

7	API Reference	20
7.1	Endpoint Summary	20
7.2	Streaming Query Route	21
7.3	Shared Route Helpers	21
7.4	Guardrails Middleware	21
7.5	Metrics Store	21
8	Configuration	22
8.1	Environment Variables	22
9	Testing and Evaluation	23
9.1	Unit Test Suite	23
9.2	Evaluation Harness	23
9.3	Query Sets	24
9.4	Recorded Results	24
10	Build, Deployment, and CI	25
10.1	Dockerfile	25
10.2	Docker Compose	25
10.3	Continuous Integration	25
10.4	Local Development Workflow	26
11	Supporting Scripts	27
11.1	scripts/download_filings.py	27
11.2	main.py	27
12	Design Decisions and Known Deviations	28

List of Figures

2.1	The six-stage agent pipeline. Auditor and Comparator run concurrently in the streaming route since neither depends on the other’s output – only on the shared <code>subtask_results</code>	6
-----	--	---

List of Tables

5.1	Section-type confidence weights as implemented in code (authoritative) versus as documented in <code>docs/ARCHITECTURE.md</code> . The code values govern actual system behavior.	16
-----	---	----

Chapter 1

Introduction and Project Purpose

1.1 Overview

FinSight (version 1.0.0) analyzes public annual reports – in the shipped corpus, FY2024–FY2026 filings for three Indian IT services companies (Infosys, TCS, and Wipro) – and answers analyst-style natural language queries such as cross-company margin comparisons, revenue-driver analysis, risk-disclosure extraction, and capital expenditure guidance comparison.

The project is deliberately engineered against a written specification (`FinSight_Project_Specification.pdf`, at the repository root) and is framed in its own design records (`docs/DECISIONS.md`) as differentiated from a "standard GenAI portfolio project": rather than a single LLM call over retrieved context, it implements a six-stage agent pipeline with hard verification gates, deterministic arithmetic where correctness matters, and full auditability.

1.2 Design Mandate

Four requirements recur throughout the documentation and the code itself, and they motivate essentially every architectural decision described in this document:

1. **Traceability.** Every factual claim in a synthesized report must carry a citation: source document, page number, and verbatim snippet.
2. **Hard blocking, not soft flagging.** Claims that cannot be entailed by retrieved evidence are removed from the final report entirely – they are not shown to the user with a caveat. This is enforced by the Auditor agent (Section 5.1.4).
3. **Cross-document reasoning as a first-class capability.** Comparing figures across multiple filings (e.g., Infosys vs. TCS, FY2024 vs. FY2025) is handled by a dedicated Comparator agent that performs *no* LLM arithmetic – all currency/unit conversion is done deterministically in Python (Section 5.2.5) before the LLM ever compares figures.
4. **Full auditability.** Every query execution persists a structured JSON audit log (Section 6.4) recording the plan, every retrieval, every claim (verified, uncertain, or blocked), and every agent invoked, with end-to-end latency.

1.3 Confidence Model

Rather than a binary answerable/unanswerable decision, FinSight assigns each claim a composite confidence score (Section 5.3.1) and classifies it into one of three tiers:

Status	Meaning	Surfaced to user?
VERIFIED	Entailed by evidence, confidence $\geq$ threshold	Yes
UNCERTAIN	Weak support or below threshold	Yes, prefixed [UNCERTAIN]
UNVERIFIABLE	Absent, contradictory, or fabricated-event pattern	No (blocked)

Chapter 2

System Architecture

2.1 Orchestration Framework

FinSight is orchestrated by Microsoft **Semantic Kernel** (SK), which the design records describe as "load-bearing" infrastructure rather than a thin wrapper around direct LLM calls. A single process-wide **Kernel** singleton is constructed by `core/sk_kernel.py:get_kernel()` and registers:

- One chat completion service (`OpenAIChatCompletion`), pointed at Groq's OpenAI-compatible endpoint (or an operator-configured fallback endpoint).
- Four **native plugins** – Python classes decorated with `@kernel_function` – for agents that perform real deterministic or LLM-hybrid computation: `RetrieverPlugin`, `AnalystPlugin`, `AuditorPlugin`, `ComparatorPlugin`.
- Two **semantic functions** (`KernelFunctionFromPrompt`, pure prompt templates rendered by the kernel) for agents that are pure text-in/text-out LLM tasks: `Planner.decompose` and `Synthesizer.synthesize`.

In practice, the Planner and Synthesizer prompt templates (`PLANNER_PROMPT`, `SYNTHESIZER_PROMPT` in `core/sk_kernel.py`) are reused as raw format strings and invoked via direct `chat_completion()` / `chat_completion_hedged()` calls in the API route handlers, rather than through `kernel.invoke()`. Only the Retriever, Analyst, Auditor, and Comparator agents are invoked through `kernel.invoke(plugin_name=..., function_name=..., arguments=KernelArguments(...))`. This mixed design – two different invocation patterns for what is conceptually one pipeline – is an explicit, documented tradeoff in `docs/DECISIONS.md`.

2.2 The Six-Stage Pipeline

1. **PlannerAgent** decomposes the user query into 2–6 subtasks (a JSON list of strings).
2. **RetrieverAgent** (run per subtask, in parallel): hybrid BM25 + dense retrieval, Reciprocal Rank Fusion, cross-encoder reranking, and confidence scoring, producing ranked chunks with citations.
3. **AnalystAgent** (per subtask): extracts KPIs and verbatim claims from the retrieved chunks via an LLM call constrained to forbid any derived/computed figure.
4. **AuditorAgent**: a single batched LLM call verifies every claim produced by every subtask via entailment checking against its supporting snippet, classifying each as VERIFIED,

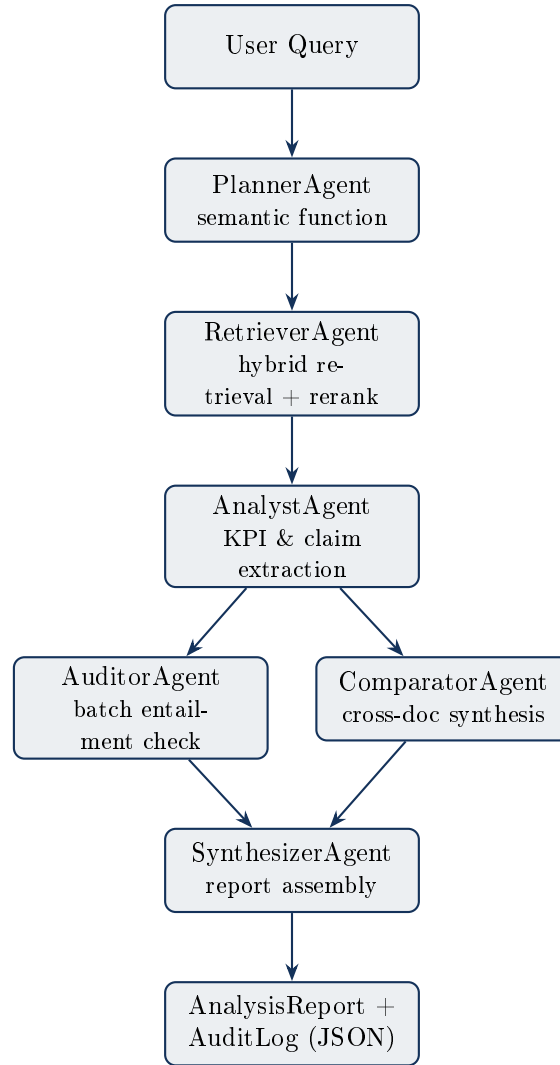


Figure 2.1: The six-stage agent pipeline. Auditor and Comparator run concurrently in the streaming route since neither depends on the other’s output – only on the shared `subtask_results`.

UNCERTAIN, or UNVERIFIABLE.

5. **ComparatorAgent** (runs concurrently with the Auditor): performs cross-document comparison of pre-normalized figures, flagging deltas exceeding a 15% deviation threshold, without performing any arithmetic itself.
6. **SynthesizerAgent**: assembles a structured Markdown report from verified and uncertain claims plus the comparison output.

2.3 Context Propagation

Citations and claims are threaded through the pipeline as JSON strings inside `KernelArguments`, so no information is silently dropped between stages:

```

user_task → subtasks (list[str]) → chunks_json → analysis_json (claims) →
claims_json → audit_json → comparison_json → final report

```


2.4 Concurrency and Latency Management

The streaming route (`api/routes/query_stream.py`) applies several latency-reduction strategies:

- All subtasks' Retriever + Analyst calls run in parallel via `asyncio.gather`.
- Auditor and Comparator run concurrently rather than sequentially, since neither consumes the other's output.
- `core/groq_client.py::chat_completion_hedged()` implements *request hedging*: the primary LLM call is fired first; if it has not returned within a configurable `hedge_after` interval (8.0 s in the streaming route), the reserve endpoint is fired concurrently and whichever completes first is used. This bounds tail (p95) latency without doubling average cost.

Chapter 3

Repository Layout

```
1  FinSight/
2  |-- agents/                               Six-stage agent pipeline (SK-orchestrated)
3  |   |-- router.py                         PlannerAgent
4  |   |-- retriever.py                     RetrieverPlugin (SK native plugin)
5  |   |-- analyst.py                      AnalystPlugin (SK native plugin)
6  |   |-- comparator.py                  ComparatorPlugin (SK native plugin)
7  |   |-- auditor.py                    AuditorPlugin (SK native plugin)
8  |   '-- synthesizer.py                 synthesize_report() (direct LLM call)
9  |-- api/                                FastAPI application
10 |   |-- main.py                         App factory, lifespan, static mounting, /
      health
11 |   |-- metrics_store.py                In-process MetricsStore singleton
12 |   |-- middleware/guardrails.py       Prompt-injection detection (pure ASGI)
13 |   |-- routes/
14 |   |   |-- query.py                    POST /query (blocking pipeline)
15 |   |   |-- query_stream.py            POST /query/stream (SSE pipeline)
16 |   |   |-- ingest.py                  POST /ingest/upload
17 |   |   |-- eval.py                    GET /eval/*
18 |   |   '-- metrics.py                  GET /metrics/json
19 |   '-- static/
20 |       |-- index.html                  Web UI (query interface + pipeline visualizer)
      )
21 |       '-- dashboard.html              Metrics dashboard
22 |-- core/                               Shared infrastructure
23 |   |-- config.py                      pydantic-settings Settings (env vars)
24 |   |-- models.py                      Dataclasses & enums
25 |   |-- sk_kernel.py                   SK Kernel builder, prompt templates
26 |   |-- groq_client.py                 Async LLM client: retry, fallback, hedging
27 |   '-- unit_normalizer.py             Deterministic INR/USD normalizer
28 |-- retrieval/                          Hybrid retrieval pipeline
29 |   |-- qdrant_store.py                 Async Qdrant client wrapper
30 |   |-- embedder.py                    SentenceTransformer wrapper
31 |   |-- bm25.py                         BM25Okapi wrapper
32 |   |-- hybrid.py                       Reciprocal Rank Fusion
33 |   |-- reranker.py                     CrossEncoder wrapper
34 |   '-- confidence.py                   5-signal composite confidence scoring
35 |-- ingestion/                          PDF -> Qdrant ingestion pipeline
36 |   |-- parser.py                       PyMuPDF page/block/span extraction
37 |   |-- chunker.py                      Heading-aware sliding-window chunker
38 |   |-- metadata.py                     Section/fiscal-year/company detection
39 |   '-- pipeline.py                     ingest_pdf(), ingest_directory()
40 |-- observability/tracer.py             structlog config, @traced decorator, OTel
      setup
41 |-- evaluation/
42 |   |-- harness.py                      Async CLI test runner
```

```
43 | | -- queries/                happy_path.json, adversarial.json
44 | | -- results/               Timestamped harness_<epoch>.json results
45 | -- tests/                  pytest unit tests
46 | -- scripts/download_filings.py Multi-strategy annual-report downloader
47 | -- data/filings/           Seed PDFs (Infosys / TCS / Wipro)
48 | -- audit_logs/             Per-run JSON audit artifacts
49 | -- qdrant_storage/         Qdrant on-disk vector data
50 | -- docs/
51 | | -- ARCHITECTURE.md        Architecture reference
52 | | -- DECISIONS.md          Design-decision records
53 | -- .github/workflows/ci.yml GitHub Actions CI (ruff + pytest)
54 | -- Dockerfile               Multi-stage build
55 | -- docker-compose.yml       qdrant + api services
56 | -- pyproject.toml           Dependencies, ruff & pytest config
```

Two files, `retrieval/chunker.py` and `retrieval/dummy_test.py`, are development-era scratch duplicates of the canonical ingestion logic used for manual PDF experimentation; they are not part of the production import graph reached by `ingestion/pipeline.py`.

Chapter 4

Technology Stack

4.1 Core Dependencies

Python ≥ 3.11 , managed with `uv`. Key libraries declared in `pyproject.toml`:

Category	Library	Role
PDF parsing	<code>pymupdf</code> (fitz)	Page/block/span text extraction
Vector database client	<code>qdrant-client</code>	Async Qdrant driver
Embeddings & reranking	<code>sentence-transformers</code>	Bi-encoder and cross-encoder models
Keyword search	<code>rank-bm25</code>	BM25Okapi sparse retrieval
Orchestration	<code>semantic-kernel</code>	Agent/plugin orchestration
LLM client	<code>openai</code>	OpenAI-compatible client pointed at Groq
LLM client (alt.)	<code>groq</code>	Native Groq SDK
API framework	<code>fastapi</code>	HTTP API
ASGI server	<code>uvicorn[standard]</code>	Application server
Configuration	<code>pydantic-settings</code>	Typed env-var settings
Logging	<code>structlog</code>	Structured logging
HTTP client	<code>httpx</code>	Async HTTP
Numerics	<code>numpy</code>	Vector math
Observability	<code>opentelemetry-*</code>	Tracing (OTLP/gRPC exporter)
Uploads	<code>python-multipart</code>	Multipart form parsing

Development dependencies: `pre-commit`, `ruff` (line-length 100, target py311, rule sets E/F/I), `pytest` + `pytest-asyncio` (`asyncio_mode="auto"`), `httpx` (for test client usage).

4.2 External Services

- **Qdrant** – vector database, collection `finsight_chunks`, 384-dimensional vectors (matching `all-MiniLM-L6-v2`), cosine distance.
- **Groq** – hosts the primary LLM, **Llama 3.3 70B** (`llama-3.3-70b-versatile`), via an OpenAI-compatible API. An optional reserve endpoint (any OpenAI-compatible provider) can be configured for hedging/failover.

4.3 Frontend

The UI (`api/static/index.html`, `dashboard.html`) is plain HTML/CSS/JS – no JavaScript framework – styled as a dark "compliance terminal" with a live pipeline visualizer that consumes the `/query/stream` SSE events, plus a separate metrics dashboard consuming `/metrics/json`.

Chapter 5

Module Reference

This chapter documents every module's responsibility, key functions, and inputs/outputs.

5.1 Agents (agents/)

5.1.1 router.py — PlannerAgent

```
1 def plan_task(user_task: str) -> list[str]:
2     ...
```

Invokes `kernel.invoke(plugin_name="Planner", function_name="decompose", arguments=KernelArguments)`. Parses the LLM's JSON list output; on JSON-parse failure falls back to bullet/numbered-list regex parsing; if that also fails, falls back to the single-element list `[user_task]`.

5.1.2 retriever.py — RetrieverPlugin

`RetrieverPlugin` exposes a single `@kernel_function(name="retrieve")` method:

```
1 def retrieve(subtask, company_filter="", fiscal_year_filter="") -> str
   :     # JSON
```

It lazily builds a BM25 index once per process (guarded by an `asyncio.Lock` with double-checked initialization) over the full Qdrant corpus, obtained via `scroll_all()`. The real work is delegated to the module-level `retrieve_chunks()`, which:

1. Filters BM25 payloads in-memory for company/fiscal-year (handling "FY2024" vs. "2024" via substring matching, since Qdrant's exact-match filter cannot).
2. Embeds the query.
3. Runs BM25 search and dense Qdrant search concurrently (`asyncio.gather`).
4. Fuses the two ranked lists via Reciprocal Rank Fusion.
5. Reranks the fused candidates via a cross-encoder.
6. Builds `RankedChunk` objects with composite confidence scores.

5.1.3 analyst.py — AnalystPlugin

```
1 def analyze(subtask, chunks_json) -> str: # JSON {"kpi's": [...], "
    claims": [...]}
```

Delegates to `analyze_chunks()`, which builds a context string from up to `settings.rerank_top_k` chunks and calls the LLM with a strict system prompt that **forbids extracting any derived or computed figure** – ratios, per-unit metrics, currency conversions, or cross-company combinations are disallowed; only verbatim printed figures may become claims. Confidence is assigned deterministically by section type (0.9 audited financials / 0.75 notes / 0.6 MD&A-or-letter). At most five claims are extracted per call. On JSON-parse failure, a regex-based fallback extraction is applied.

5.1.4 auditor.py — AuditorPlugin

```
1 def audit(claims_json, confidence_threshold="0.65", original_query="")
    -> str:
```

Core logic lives in `audit_claims()`. Claims are split into two groups:

- Claims *with* a supporting snippet: batch-audited in a **single** LLM call (`_batch_entailment()`) for efficiency and consistency.
- Claims *without* a supporting snippet: instantly classified **UNVERIFIABLE** with reason "No supporting snippet provided" – no LLM call needed.

The batch-entailment prompt applies two checks, in order:

1. **Fabricated-event detection:** if the original query names a specific external company plus an action verb, and the claim's topic is unrelated, the claim is forced to **UNVERIFIABLE** regardless of superficial snippet overlap. This targets adversarial queries such as "what did the CEO say about acquiring Google."
2. **Standard entailment classification:** **VERIFIED** if the snippet entails the claim and confidence $\geq$ threshold; **UNCERTAIN** if support is weak or confidence is below threshold; **UNVERIFIABLE** if the snippet is absent or contradicts the claim.

Returns a triple (verified, uncertain, unverifiable) of `AuditedClaim` lists.

5.1.5 comparator.py — ComparatorPlugin

```
1 def compare(subtask_results_json, original_query) -> str:
```

Delegates to `compare_results()`, which calls the LLM under a system prompt that **prohibits any arithmetic**: no per-unit metrics, no currency/scale conversion, no percentage-delta computation, no cross-company arithmetic. The LLM's sole job is to place pre-normalized (already converted to Rs. crore) verbatim values side by side and flag anomalies – deviations exceeding 15% between comparable figures, unit/currency mismatches, or contradictions with auditor statements. Output:

```
1 {"deltas": [...], "cross_document_claims": [...], "summary": "..."}

```

5.1.6 synthesizer.py — SynthesizerAgent

```
1 def synthesize_report(query, verified_claims, uncertain_claims,
```

2

`comparison, task_id) -> str:`

A direct `chat_completion()` call (not SK tool-calling) using the `SYNTHESIZER_PROMPT` template from `core/sk_kernel.py`. Returns an "insufficient evidence" message outright if there are no verified or uncertain claims. The prompt enforces: only verbatim figures with in-line citations; an `[UNCERTAIN]` prefix on uncertain claims; omission of derived figures unless pre-labeled `[converted from ...]`; no arithmetic; explicit N/A for missing data; and a fixed GitHub-Flavored-Markdown section structure (Executive Summary / Key Findings / Comparative Analysis / Risk Flags).

5.2 Core Infrastructure (`core/`)

5.2.1 `config.py`

`Settings(BaseSettings)` – a `pydantic-settings` model reading `.env`. All fields are enumerated in Table 8.1.

5.2.2 `models.py`

The canonical dataclasses and enums shared across the entire system; see Chapter 6.

5.2.3 `sk_kernel.py`

Builds and holds the SK Kernel singleton (`get_kernel()`), an optional reserve kernel (`get_fallback_kernel()`) and `reset_kernel()`. Contains the `PLANNER_PROMPT` and `SYNTHESIZER_PROMPT` string templates and registers native plugins plus semantic functions inside `_build_kernel()`.

5.2.4 `groq_client.py`

Lazily-initialized singleton `AsyncOpenAI` clients (primary, and optionally a reserve client selected when fallback environment variables are set).

- `chat_completion()`: retries on `RateLimitError` / HTTP 503 with backoff [1.0, 2.0, 4.0] seconds; breaks immediately to the reserve client on timeout or connection errors; raises `RuntimeError` if both primary and reserve fail.
- `chat_completion_hedged()`: implements the hedged dual-request pattern described in Section 2.2 – fires the primary request, and after a configurable delay fires the reserve concurrently, returning whichever completes first.

5.2.5 `unit_normalizer.py`

A pure-Python, regex-driven, deterministic currency/unit normalizer: `normalize_text()` and `normalize_subtask_results()`. Converts expressions such as `$X billion/million/...` and `Rs/INR X crore/lakh/million/...` into a canonical `Rs. X crore [converted from ...]` form using `Decimal` arithmetic (`USD_TO_INR = 84`). This module exists specifically so that the Comparator and Synthesizer LLM stages never perform currency or scale arithmetic themselves – all such conversion happens deterministically in Python before the LLM sees the data, eliminating a whole class of hallucinated arithmetic.

5.3 Retrieval Pipeline (`retrieval/`)

`qdrant_store.py` `QdrantStore` wraps `AsyncQdrantClient`: `ensure_collection()` (384-dim, cosine distance), `upsert_chunks()`, `dense_search()` (with optional payload `Filter`), `scroll_all()` (paginated full-corpus fetch feeding the BM25 index), `delete_by_doc_id()`, `collection_info()`.

`embedder.py` `Lazy SentenceTransformer`(all-MiniLM-L6-v2) singleton; `embed_query()`, `embed_texts()` (normalized embeddings).

`bm25.py` `BM25Retriever` wraps `rank_bm25.BM25Okapi`; a custom `_tokenize()` regex deliberately keeps % attached to its number (so 20.7% tokenizes as one token) since financial figures with percent signs are common retrieval targets.

`hybrid.py` `reciprocal_rank_fusion`(`ranked_lists`, `k=60`) – a purely rank-based fusion that is invariant to the differing score scales of BM25 and cosine similarity.

`reranker.py` `Lazy CrossEncoder`(ms-marco-MiniLM-L6-v2) singleton; `rerank(query, candidates, top_k)` replaces stage-1 fused scores entirely with cross-encoder relevance scores.

`confidence.py` `compute_confidence()` and `build_ranked_chunks()`; see Section 5.3.1.

5.3.1 Confidence Scoring

`compute_confidence()` computes a weighted composite of five signals:

Signal	Weight	Description
Retrieval score	35%	Cross-encoder rerank score
Section-type weight	25%	Static weight by document section (see Table 5.1)
Freshness	15%	Decays 0.2/year from a 2025 baseline, floor 0.2
Cross-filing consistency	15%	1.0 if ≥ 3 same-company docs corroborate, 0.75 if 2, else 0.4
Retrieval consistency	10%	Computed from <code>query_variants_hit</code> (placeholder signal)

5.4 Ingestion Pipeline (`ingestion/`)

`parser.py` `parse_pdf()` (page/block dictionaries), `extract_text_with_positions()` (span-level with bounding boxes), `get_page_text()` – all via PyMuPDF.

`chunker.py` `chunk_document(pages, doc_id, source, target_tokens=400, overlap_tokens=80)`: a heading-aware sliding window. Headings are detected via a font-size heuristic (`span.size > body_size * 1.2`); a new chunk starts at a heading boundary once enough tokens have accumulated, otherwise the window slides with overlap. `chunk_id` is a SHA-256 hash of (`doc_id`, `page`, `index`, `text[:100]`). `_deduplicate()` drops chunks whose first 200 characters hash identically, keeping the first occurrence.

`metadata.py` `detect_section_type()` (regex cascade: audited financials $\rightarrow$ notes $\rightarrow$ MD&A $\rightarrow$ letter $\rightarrow$ unknown, with a "note" sub-check redirecting some audited-financials matches to NOTES), `detect_fiscal_year()` (regex `FY\d{4}`, falling back to a bare-year scan 2024 $\rightarrow$ 2020), `detect_company()` (filename plus first-200-character text hints for Infosys/TCS/Wipro, handling underscore/hyphen normalization and a "Tata Consultancy" alias), `section_type_confidence_weight()` (weights 1.0 / 0.85 / 0.65 / 0.40 / 0.50 – see note in Table 5.1).

`pipeline.py ingest_pdf(pdf_path, doc_id=None, overwrite=True):` parse → chunk → batch-embed → `ensure_collection()` → optional `delete_by_doc_id()` (for re-ingestion idempotency) → `upsert_chunks()`. `ingest_directory(directory, pattern="*.pdf")`: batch-ingests every matching file, capturing per-file errors without aborting the batch.

Table 5.1: Section-type confidence weights as implemented in code (authoritative) versus as documented in `docs/ARCHITECTURE.md`. The code values govern actual system behavior.

Section type	Code weight	Docs-stated weight
Audited financials	1.00	1.0
Notes	0.85	0.8
MD&A	0.65	0.7
Letter	0.40	0.5
Unknown	0.50	0.4

5.5 Observability (`observability/tracer.py`)

`setup_tracing()` is idempotent (guarded by a `_configured` flag) and configures `structlog` with either a `ConsoleRenderer` (text format) or `JSONRenderer` (JSON / OpenTelemetry-enabled format), merging context variables and adding log level plus ISO timestamp. When `OTEL_ENABLED` is set, an OpenTelemetry `TracerProvider` with an `OTLPSpanExporter` is additionally configured. The `traced(name=None)` decorator logs start/complete/error events with millisecond latency for any decorated async function.

Chapter 6

Data Models

All models are defined in `core/models.py` as dataclasses (from `__future__` import `annotations`).

6.1 Enumerations

```
1 class SectionType(str, Enum):
2     AUDITED_FINANCIALS = "audited_financials"
3     MDA = "mda"
4     NOTES = "notes"
5     LETTER = "letter"
6     UNKNOWN = "unknown"
7
8 class AuditStatus(str, Enum):
9     VERIFIED = "verified"
10    UNCERTAIN = "uncertain"
11    UNVERIFIABLE = "unverifiable"
```

6.2 Core Dataclasses

```
1 @dataclass
2 class Citation:
3     document: str
4     page: int
5     snippet: str
6     claim: str
7     confidence: float
8     section_type: str
9     # to_dict()
10
11 @dataclass
12 class Chunk:
13     chunk_id: str
14     doc_id: str
15     source: str
16     text: str
17     page: int
18     section: str
19     section_type: SectionType
20     token_count: int
```

```

21     fiscal_year: str = ""
22     company: str = ""
23     # to_payload() / from_payload() -- Qdrant payload (de)
        serialization
24
25 @dataclass
26 class RankedChunk:
27     chunk: Chunk
28     retrieval_score: float
29     confidence_score: float
30     # to_citation(claim="") -> Citation
31
32 @dataclass
33 class AuditedClaim:
34     claim: str
35     citation: Citation
36     audit_status: AuditStatus
37     audit_reason: str
38     # to_dict()
39
40 @dataclass
41 class SubtaskResult:
42     subtask: str
43     ranked_chunks: list[RankedChunk]
44     kpis: list[dict]
45     claims: list[AuditedClaim]
46     agents_used: list[str] = field(default_factory=list)
47
48 @dataclass
49 class AuditLog:
50     task_id: str
51     timestamp: str
52     user_query: str
53     plan: list[str]
54     retrievals: dict[str, list[str]]
55     claims: list[dict]
56     flagged_uncertain: list[str]
57     blocked_unverifiable: list[str]
58     agents_invoked: list[str]
59     latency_ms: int
60     # to_dict()
61
62 @dataclass
63 class AnalysisReport:
64     task_id: str
65     query: str
66     summary: str
67     verified_claims: list[AuditedClaim]
68     uncertain_claims: list[AuditedClaim]
69     audit_log: AuditLog
70     # to_dict()

```

6.3 Qdrant Collection Schema

Collection `finsight_chunks`: 384-dimensional vectors (all-MiniLM-L6-v2), cosine distance.
 Payload fields: `chunk_id`, `doc_id`, `source`, `text`, `page`, `section`, `section_type`, `token_count`,

`fiscal_year`, `company`. Payload-level filtering is used for `company` during dense search directly in Qdrant; fiscal-year filtering is instead performed in-memory over the BM25 payload set, since Qdrant's filter requires exact string matches and fiscal-year expressions need substring/"FY"-prefix handling.

6.4 Audit Log Example

The following excerpt (structurally faithful to a real audit log, `audit_logs/0a0601db-...json`, for the query *"revenue per employee of all the companies"*) illustrates the schema in practice:

```
1 {
2   "task_id": "0a0601db-...",
3   "timestamp": "2026-...",
4   "user_query": "revenue per employee of all the companies",
5   "plan": ["...", "..."],
6   "retrievals": {"subtask 1": ["chunk_id_a", "chunk_id_b"]},
7   "claims": [ { "claim": "...", "citation": {...},
8                 "audit_status": "uncertain",
9                 "audit_reason": "..." } ],
10  "flagged_uncertain": ["..."],
11  "blocked_unverifiable": [],
12  "agents_invoked": ["planner", "retriever", "analyst",
13                    "auditor", "comparator", "synthesizer"],
14  "latency_ms": 50466
15 }
```

Notably, in this real run the AuditorAgent correctly classified the derived-metric claim "revenue per employee for Infosys $\approx$ Rs. 453.1 lakh" as **uncertain** – the supporting snippet confirms revenue but not headcount, so the ratio cannot be verified from the cited evidence – rather than either fabricating verification or silently accepting the LLM's arithmetic. The Planner also over-expanded this query to include companies absent from the corpus (HCL, Cognizant); those subtasks correctly retrieved no evidence and produced no spurious verified claims, evidencing that the hard-blocking design works as intended under realistic query drift.

Chapter 7

API Reference

7.1 Endpoint Summary

Route	Method	Handler file	Description
/query	POST	api/routes/query.py	Blocking full pipeline. Body: {query, company_filter?, fiscal_year_filter?, confidence_threshold?}. Returns AnalysisReport.to_dict().
/query/stream	POST	api/routes/query_stream.py	Server-Sent-Events streaming variant; events: start, planned, retrieved, analyzed, audited, compared, done, error.
/ingest/upload	POST	api/routes/ingest.py	Multipart PDF upload (file=@x.pdf), optional doc_id, overwrite (default true). Triggers ingest_pdf().
/eval/collection	GET	api/routes/eval.py	Qdrant collection stats (name, vector/point counts, status).
/eval/audit-logs	GET	api/routes/eval.py	Lists up to 20 most recent audit log filenames plus total count.
/eval/audit-logs/{log_id}	GET	api/routes/eval.py	Returns a specific audit log JSON (tolerant of a missing .json suffix).
/metrics/json	GET	api/routes/metrics.py	Full metrics snapshot.
/health	GET	api/main.py	Liveness probe: {"status": "ok", "model": <groq_model>}.
/ui, /ui/	GET	api/main.py	Serves the web UI (no-cache headers); static files mounted beneath.
/dashboard	GET	api/main.py	Serves the metrics dashboard.
/	GET	api/main.py	Redirects to /ui.

All routes are protected globally by `GuardrailsMiddleware` (POST/PUT bodies scanned for prompt-injection patterns) and `CORSMiddleware` (`allow_origins=["*"]`). Interactive Swagger documentation is auto-served by FastAPI at `/docs`.

7.2 Streaming Query Route

`api/routes/query_stream.py` defines the SSE variant of the pipeline. It emits an ordered sequence of named events as each stage completes:

`start` → `planned` → `retrieved*` → `analyzed*` → `audited` → `compared` → `done` (or error at any point)

(*emitted once per subtask). Subtasks are run in parallel via a `_run_subtask()` helper; the Planner call uses `chat_completion_hedged()`.

7.3 Shared Route Helpers

`api/routes/query.py` defines a `QueryRequest` Pydantic model (`query`, `company_filter`, `fiscal_year_filter`, `confidence_threshold`) and several helpers reused by both the blocking and streaming routes: `_parse_subtasks()`, `_parse_audit_result()`, `_safe_confidence()`, `_deserialize_claims()`, and `_save_audit_log()` (writes to `settings.audit_log_dir/<task_id>.json`).

7.4 Guardrails Middleware

`GuardrailsMiddleware` (`api/middleware/guardrails.py`) is implemented as **pure ASGI** middleware rather than `BaseHTTPMiddleware`. It buffers the full POST/PUT request body and scans it against a set of prompt-injection patterns (instructions to ignore/forget prior context, "you are now", "jailbreak", `<script>` tags, `[system]` markers, etc.), rejecting matches with an HTTP 400 JSON response. When no match is found, the buffered body is replayed to the first downstream `receive()` call, and subsequent calls delegate to the real `receive`. This specific replay design exists to remain compatible with SSE streaming responses, which `BaseHTTPMiddleware` cannot support without buffering the entire response.

7.5 Metrics Store

`MetricsStore` (`api/metrics_store.py`) is a GIL-protected in-process singleton (module-level instance `metrics`) tracking:

- Total queries, errors, and in-flight request counts.
- A rolling 200-sample latency deque, exposing p50/p95/average/min/max plus fixed latency buckets.
- Per-agent rolling 100-sample latency deques for all six agent stages.
- A 50-entry recent-queries ring buffer and a 20-entry recent-errors ring buffer.
- Running totals of verified / uncertain / blocked claims.

Its `snapshot()` method produces the full JSON body served at `/metrics/json`.

Chapter 8

Configuration

8.1 Environment Variables

`core/config.py::Settings` is a `pydantic-settings` model (`extra="ignore"`) reading `.env`. `.env.example` mirrors this table exactly.

Variable	Default	Purpose
<code>groq_api_key</code>	""	Groq API key (required in practice)
<code>groq_model</code>	<code>llama-3.3-70b-versatile</code>	Primary LLM
<code>groq_base_url</code>	<code>https://api.groq.com/openai/v1</code>	OpenAI-compatible endpoint
<code>fallback_api_key</code>	""	Reserve LLM API key
<code>fallback_model</code>	""	Reserve LLM model name
<code>fallback_base_url</code>	""	Reserve LLM endpoint (any OpenAI-compatible service)
<code>qdrant_host</code>	<code>localhost</code> (qdrant in Docker)	Qdrant host
<code>qdrant_port</code>	<code>6333</code>	Qdrant HTTP port
<code>qdrant_collection</code>	<code>finsight_chunks</code>	Collection name
<code>embedding_model</code>	<code>sentence-transformers/Baichuan2-L6-v2</code>	Embedding model
<code>reranker_model</code>	<code>cross-encoder/ms-marco-MiniLM-L6-v2</code>	Cross-encoder
<code>chunk_size</code>	<code>400</code>	Target chunk size (tokens)
<code>chunk_overlap</code>	<code>80</code>	Overlap (tokens)
<code>retrieval_top_k</code>	<code>20</code>	Candidates per retriever
<code>rerank_top_k</code>	<code>5</code>	Final chunks after reranking
<code>confidence_threshold</code>	<code>0.65</code>	VERIFIED cutoff
<code>hallucination_fallback_threshold</code>	<code>0.55</code>	UNCERTAIN floor
<code>audit_log_dir</code>	<code>audit_logs</code>	Per-run audit log output directory
<code>otel_enabled</code>	<code>false</code>	Enable OpenTelemetry export
<code>otel_endpoint</code>	<code>http://localhost:4317</code>	OTLP gRPC endpoint
<code>log_level</code>	<code>INFO</code>	Log verbosity
<code>log_format</code>	<code>text</code>	<code>text</code> (console) or <code>json</code> (structured)

Docker Compose overrides `QDRANT_HOST=qdrant`, `LOG_FORMAT=json`, and `LOG_LEVEL=INFO` for the `api` service, loading secrets via `env_file: .env`.

Chapter 9

Testing and Evaluation

9.1 Unit Test Suite

Framework: **pytest** + **pytest-asyncio** (`asyncio_mode = "auto"`), `testpaths = ["tests"]`. All tests are pure unit tests of deterministic logic with no live dependency on Qdrant or Groq.

`test_chunker.py` `_make_chunk_id()` determinism/uniqueness; `_deduplicate()` behavior (exact-duplicate removal, order preservation, 200-character-prefix truncation edge case).

`test_confidence.py` `_freshness_score()`, `_cross_filing_score()`, `compute_confidence()` (bounds, section-type ordering, rerank-score precedence), `build_ranked_chunks()`.

`test_ingestion.py` `detect_fiscal_year()`, `detect_section_type()` (financials/MD&A/letter/notes/unknown), `detect_company()` (including the TCS alias).

`test_models.py` Dataclass round-trips: `Citation.to_dict()`, `Chunk.to_payload()/from_payload()`, `AuditedClaim` for every `AuditStatus`, default-field handling.

`test_retrieval.py` `_tokenize()`, `BM25Retriever.search()`, `reciprocal_rank_fusion()` (including an exact RRF score assertion of 2/61 and rank-1-vs-rank-2 tie behavior).

`test_unit_normalizer.py` 30+ cases covering `normalize_text()` / `normalize_subtask_results()`: USD billion/million/thousand/trillion/bn conversions, INR crore/lakh/million/billion conversions, idempotency on already-converted labels, non-mutation of input dicts, and edge cases (zero, decimals, Indian comma grouping, mixed currencies within one sentence).

No integration tests exercise the live FastAPI app, Qdrant, or Groq directly within `tests/` – that role is filled instead by the evaluation harness described below.

9.2 Evaluation Harness

`evaluation/harness.py::run_harness(query_file)` is an end-to-end scoring tool distinct from the pytest suite: it drives the *live* `/query` endpoint (`http://localhost:8000/query`, 360 s timeout) rather than mocking it. Each test case carries an `expected_behavior` label, and `score_result()` applies behavior-specific pass/fail logic:

expected_behavior	Passes when
unverifiable	All claims blocked / no verified claims produced
unverifiable_or_insufficient_evidence	Blocked, or insufficient-evidence response
uncertain_or_unverifiable	Claims are uncertain or blocked (none false)
verified_or_uncertain / partial_or_insufficient_evidence (default)	At least one verified claim, or an explicit partial response Requires ≥ 1 verified claim

Each run writes a timestamped `harness_<epoch>.json` summary to `evaluation/results/`.

9.3 Query Sets

`evaluation/queries/happy_path.json` 4 cases targeting verifiable facts: Infosys margin, Infosys/TCS revenue-growth comparison, Wipro revenue drivers, TCS headcount – all **expected_behavior: verified_or_uncertain**.

`evaluation/queries/adversarial.json` 4 hallucination-eliciting cases: a future-year query (FY2030, no data can exist), a fabricated event ("what did the CEO say about acquiring Google"), a cross-domain query (Microsoft/Apple, absent from the corpus), and a numerically precise but implausibly-stated figure (exact free cash flow to the rupee).

9.4 Recorded Results

`evaluation/results/` contains 12 historical run artifacts. A representative happy-path run (`harness_1783872558.json`) reports a **4/4 pass rate (1.0)**, with per-test latency ranging 7.6–26.3 s – e.g., the cross-company revenue-growth comparison query returned 17 verified claims in 22.9 s. The project README states currently published scores of **4/4 on the happy-path set and 3/4 on the adversarial set**, the fourth adversarial case intentionally timing out by design on a five-company cross-domain query that forces excessive subtask fan-out.

Chapter 10

Build, Deployment, and CI

10.1 Dockerfile

A two-stage build:

1. **Builder stage** (`python:3.11-slim`): installs `uv`, extracts runtime dependencies from `pyproject.toml` via the standard-library `tomllib`, creates a `.venv`, and installs dependencies into it. This layer only rebuilds when `pyproject.toml` or `uv.lock` change.
2. **Runtime stage** (`python:3.11-slim`): installs only `curl` (used by the healthcheck), copies the pre-built `.venv` from the builder stage (no `pip/uv/build` toolchain present in the final image), copies application source (respecting `.dockerignore`), creates `audit_logs/` and `data/filings/`, and runs as a non-root user `finsight`. Exposes port 8000 and defines a `HEALTHCHECK` against `/health` with a 60s start period (to accommodate cold-start model downloads). Entrypoint: `uvicorn api.main:app -workers 1 -access-log -log-level warning` – a single worker is deliberate, preserving the in-process SK kernel singleton and metrics store across requests.

10.2 Docker Compose

Two services:

qdrant Image `qdrant/qdrant:latest`; ports 6333/6334; volume-mounts `./qdrant_storage`; TCP healthcheck; json-file logging (20 MB × 3 files).

api Built from the local `Dockerfile`; port 8000; environment overrides for Docker networking (`QDRANT_HOST=qdrant`, `LOG_FORMAT=json`); secrets loaded via `env_file: .env`; volume-mounts `./audit_logs` and `./data`; `depends_on: qdrant: condition: service_healthy`; json-file logging (50 MB × 5 files); its own HTTP healthcheck.

10.3 Continuous Integration

`.github/workflows/ci.yml` (GitHub Actions), triggered on push/PR to `main`: sets up Python 3.11 and `uv`, runs `uv sync -group dev`, then `ruff check .`, `ruff format -check .`, and finally `pytest tests/ -v -tb=short`. No separate Docker build/publish or continuous-deployment job is present.

10.4 Local Development Workflow

1. `docker compose up qdrant -d` – start only the vector database.
2. `uv sync` – install dependencies locally.
3. `uvicorn api.main:app -reload` – run the API with hot reload outside Docker for fast iteration.

`.dockerignore` excludes `.env`, `qdrant_storage`, `tests`, and `__pycache__` (among others) from the Docker build context and final image.

Chapter 11

Supporting Scripts

11.1 `scripts/download_filings.py`

A multi-strategy downloader for the seed corpus of annual-report PDFs, attempting each strategy in order until one succeeds:

1. The BSE India filing API, queried by scrip code.
2. A table of known direct PDF URL hints.
3. HTML scraping of investor-relations pages for `.pdf` links matching the target fiscal year.
4. An interactive browser fallback that opens the page and waits for a manual save, for filings that cannot be located programmatically.

Downloads are validated against a minimum file size and expected content type, and a final summary of successes/failures is reported.

11.2 `main.py`

A trivial placeholder entrypoint (`print("Hello from finsight!")`) retained from project scaffolding; it is *not* the real application entrypoint. The actual service is served via `uvicorn api.main:app`.

Chapter 12

Design Decisions and Known Deviations

This chapter records intentional design tradeoffs and points where implementation details diverge from the idealized descriptions in `docs/ARCHITECTURE.md`, as verified directly against the source.

1. **Semantic Kernel is load-bearing, not decorative.** Four agents are genuine native SK plugins; only the Planner and Synthesizer are pure prompt templates, and even those are invoked as raw `chat_completion()` calls reusing the SK-authored prompt strings rather than through `kernel.invoke()`. This mixed invocation style is acknowledged in `docs/DECISIONS.md` as a deliberate, if inelegant, tradeoff.
2. **Auditor/Comparator concurrency is an implementation-level optimization** beyond what the architecture diagram literally shows (which draws Comparator strictly after Auditor); the code recognizes both depend only on `subtask_results` and are safe to parallelize.
3. **Section-type confidence weights in code differ slightly from documented values** (Table 5.1); the code is authoritative for actual system behavior.
4. **All currency/unit arithmetic is deterministic**, performed in `core/unit_normalizer.py` before either the Comparator or Synthesizer LLM stage sees the data – explicitly to prevent LLM arithmetic hallucination on financial figures.
5. **Hard blocking over soft flagging.** The Auditor’s UNVERIFIABLE classification removes a claim from the synthesized report entirely; it is never surfaced to the end user with a disclaimer, unlike the UNCERTAIN tier.
6. **retrieval/chunker.py and retrieval/dummy_test.py are non-canonical.** They duplicate ingestion logic for manual/ad hoc PDF experimentation during development and are not reachable from the production ingestion pipeline (`ingestion/pipeline.py`).